Axcelson O. Macansantos
BSIT-3 / IT342 – G4

April 6, 2026

APPLICATION OF SOFTWARE DESIGN PATTERNS

RESEARCH Phase:

RESEARCH Phase:

Creational #1 — Factory Pattern

Problem: Object creation gets messy fast. When you're manually building objects in multiple places, forgetting to update one of them causes bugs that are hard to trace.

How it works: You hand off the construction job to one dedicated factory class. You just call it, and it gives you a ready-to-use object no need to worry about the steps involved.

Real-world example: Stripe uses this when building payment objects. A factory figures out whether you're paying by card, bank transfer, or wallet — the rest of the code doesn't have to care.

In CleanIT: BookingFactory and UserFactory handle building those entities from raw request data. No more repetitive setter chains scattered around the controller and service layers.

Creational #2 — Builder Pattern

Problem: Objects with lots of fields often get assembled piece by piece across different parts of the code. Miss one setter and you've got a broken object with no obvious error.

How it works: A Builder inner class lets you chain all the fields together in one readable statement, then call .build() at the end.

Real-world example: OkHttp does this for HTTP requests — .url().header().post().build(). Clean, readable, hard to mess up.

In CleanIT: LoginResponse was being built the same way in three different places. Adding a Builder class cleaned that up into one consistent, readable chain everywhere it's used.

Structural #1 — Facade Pattern

Problem: Controllers that dig directly into multiple repositories end up doing too many things at once — they're handling HTTP and acting as a data layer. That makes them painful to test or change.

How it works: A Facade class sits in between and exposes simple, meaningful methods. The controller just calls one thing and the facade handles all the behind-the-scenes coordination.

Real-world example: Booking.com's reservation flow internally talks to inventory, payments, and email — but from the frontend's perspective, it's just one call.

In CleanIT: BookingFacade handles both BookingRepository and UserRepository so the controller doesn't have to. BookingController went from 183 lines with 2 repo dependencies down to 75 lines with just 1.

Structural #2 — Decorator Pattern

Problem: Stuffing logging or caching directly into a service class mixes concerns. It bloats the class and makes that behavior impossible to remove without touching business logic.

How it works: A Decorator wraps the real class, implements the same interface, and adds behavior before or after each call — completely invisible to whoever's calling it.

Real-world example: Java's BufferedInputStream wraps FileInputStream to add buffering without changing how you use it. Same idea.

In CleanIT: LoggingBookingService wraps BookingFacade and logs every operation automatically. It's marked @Primary so Spring injects it wherever the facade is needed — and BookingController has zero logging code of its own.

Behavioral #1 — Strategy Pattern

Problem: Hardcoding validation rules inside a controller works until requirements change. Then you're editing core logic just to add a new rule which is exactly what you want to avoid.

How it works: You define an interface for the rule, then write separate classes for each variation. The component just depends on the interface and doesn't care which strategy it gets.

Real-world example: E-commerce shipping StandardShipping, ExpressShipping, FreightShipping all implement the same interface. The order service picks the right one at runtime, no ugly if-else chains needed.

In CleanIT: BookingStatusValidator replaces the inline status-check list that was sitting in the controller. Later, a RoleBasedStatusValidator for technicians vs. clients can be swapped in without touching BookingFacade or BookingController at all.

Behavioral #2 — Observer Pattern

Problem: Triggering side effects (notifications, emails, analytics) directly inside an update method creates tight coupling. Every new feature means editing core logic — risky and hard to manage.

How it works: The subject just fires an event when something happens. Any number of listeners subscribe and react independently, without the subject knowing they exist.

Real-world example: When a Grab trip status changes, separate listeners handle the SMS, the ETA update, and the analytics log — the booking service fires the event and forgets about it.

In CleanIT: After BookingFacade.updateBookingStatus() saves the change, it publishes a BookingStatusChangedEvent. BookingNotificationListener catches it and logs a notification. Want to add email later? Just add another listener — nothing else changes..

REFACTORING Phase

Repository: AksilDev/IT342_CleanIT_G4_Macansantos **Branch:** feature/design-patterns-refactor *(not merged into main)*

Branch	Updated	Check status	Behind	Ahead	Pull request
feature/design-patterns-refactor	1 hour ago		3	1	

Commit cec68e2

Browse files

AksilDev committed 1 hour ago

Applied Factory Pattern to BookingFactory and UserFactory

feature/design-patterns-refactor

1 parent 203d9e9 commit cec68e2

Filter files...

- backend/src/main/java/com/...
  - controller
    - AuthController.java
    - BookingController.java
  - dto
    - LoginResponse.java
  - event
    - BookingNotificationListen...
    - BookingStatusChangedEve...
  - facade
    - BookingFacade.java
  - factory
    - BookingFactory.java
    - UserFactory.java
  - service
    - AuthService.java
    - BookingService.java
    - LoggingBookingService.java
  - strategy
    - BookingStatusValidator.java
    - DefaultStatusValidator.java

13 files changed +835 -184 lines changed

Search within code

backend/src/main/java/com/GA/backend/controller/AuthController.java +19 -20

```
@@ -18,6 +18,11 @@
 import java.util.Map;
 import java.util.UUID;

+/**
+ * AuthController — refactored to use:
+ * Builder Pattern → LoginResponse.Builder replaces raw setter block
+ * in completeOAuthRegistration
+ */
 @RestController
 @RequestMapping("/api/v1/auth")
 @CrossOrigin(origins = "http://localhost:5173")
@@ -27,7 +32,8 @@ public class AuthController {
     private final UserRepository userRepository;
     private final SupabaseStorageService storageService;

-    public AuthController(AuthService authService, UserRepository userRepository, SupabaseStorageService
storageService) {
+    public AuthController(AuthService authService, UserRepository userRepository,
+            SupabaseStorageService storageService) {
         this.authService = authService;
         this.userRepository = userRepository;
         this.storageService = storageService;
@@ -36,10 +42,9 @@ public AuthController(AuthService authService, UserRepository userRepository, Su
     @PostMapping("/upload-image")
     public ResponseEntity<?> uploadImage(@RequestParam("file") MultipartFile file) {
         try {
-            // Generate temporary user ID for the filename
```